Test Coverage – Course and Tutor Registration Feature

Application: MindBridge AI Tutor

Prepared By: [Əzizova Gültəkin]

Date: [25.09.2026]

Version: 1.0

Feature Scope: Multi-step course and tutor registration workflow, including: session browsing, cart management, and mock payment checkout.

Test Coverage Table

Test Type	Description	Environment	Environment Justification	Pass/Fail Criteria
Functional Testing	Verifies that the end-to-end registration workflow — browsing sessions, adding a session to the cart, and completing checkout — works correctly from start to finish.	QA (localhost:8081)	Functional testing requires a fully operational application and a populated test database, but does not require real student PII. The QA environment's scrubbed database provides realistic test data without exposing sensitive information, making it the appropriate and safe environment for this test type.	A student can browse courses, add a session to the cart, complete checkout, and receive an order confirmation with the correct total, with no errors at any step.
Negative / Error Handling Testing	Confirms the system correctly rejects or safely handles invalid input, including the known defect where a special character (e.g., @) in the payment field bypasses front-end validation and triggers an HTTP 500 error.	QA (localhost:8081)	This testing intentionally submits invalid and malformed data, which could corrupt or expose data if run against realistic PII. The QA environment's anonymized data set allows testers to safely trigger and document error conditions, such as the HTTP 500 defect, without any risk to real or realistic student information.	Invalid input (special characters, empty fields, malformed card numbers) is rejected with a clear, user-facing error message, and no unhandled server errors (e.g., HTTP 500) occur; any that do are logged as defects.
Regression Testing	Re-executes core registration test cases after a bug fix or code change to confirm that the fix did	QA (localhost:8081)	Regression testing is run frequently, after nearly every fix or build, so it needs an environment	All previously passing test

Test Type	Description	Environment	Environment Justification	Pass/Fail Criteria
	not break previously working functionality.		that is fast to reset and safe to use repeatedly. The QA environment's scrubbed database supports this frequent re-testing without any risk of exposing or corrupting realistic student PII.	cases in the registration workflow (browse, cart, checkout) still pass after the fix, with no new defects introduced.
Usability Testing	Assesses whether a real student can intuitively complete the multi-step registration process (browsing, cart, checkout) without confusion or unnecessary friction.	STAGING (localhost:8082)	Usability is best judged against content and data that closely resemble what a real student will actually see and experience, so it should be evaluated in a production-like environment. STAGING's unscrubbed, realistic simulated data gives testers an accurate impression of the real student experience, which the anonymized QA data cannot fully replicate.	A tester unfamiliar with the feature can browse, add a session to the cart, and complete checkout without external guidance, encountering no confusing or misleading UI elements.
Responsive Design Testing	Verifies the registration workflow displays and functions correctly across screen sizes, including the known defect where the primary navigation overlaps main content below a 400px viewport width.	QA (localhost:8081) and STAGING (localhost:8082)	Initial responsive checks (layout, overlap, tap targets) only require a working application and can be safely and repeatedly performed in QA using anonymized data. Final validation should also be confirmed in STAGING, since it is the production-like environment and confirms the layout behaves correctly under realistic, production-representative conditions before release.	The registration workflow remains fully usable and free of overlapping or inaccessible elements across standard breakpoints, including viewports below 400px width.

Known Defects Affecting This Feature

Defect ID: DEF-001

Description: Entering a special character (for example, @) into the payment field bypasses front-end validation and causes the application to return an HTTP 500 Internal Server Error.

Visible In: Browser Network tab

Recommended Test Environment: QA (localhost:8081)

Reason: This defect can be safely reproduced and documented in the QA environment using anonymized test payment data. Testing in QA eliminates any risk to real user accounts or financial systems while allowing developers to investigate the issue.

Defect ID: DEF-002

Description: When the browser viewport width is reduced below 400px (mobile emulation), the primary navigation menu overlaps the main content area, making it impossible for the user to interact with the page.

Visible In: User interface (browser viewport, when resized or emulating a mobile device)

Recommended Test Environment: QA (localhost:8081)

Reason: This is a layout/rendering defect that can be fully reproduced and observed using the browser's device emulation tools against the QA environment's anonymized test data, with no need for realistic PII. Using QA allows testers and developers to safely inspect and iterate on the fix without touching production-like or sensitive data.